

1 Abstract

This report details the quantitative analysis of High-Performance Liquid Chromatography (HPLC) data aimed at establishing a correlation between integrated peak areas and concentration estimates. The workflow involved rigorous signal preprocessing, including Asymmetric Least Squares (AsLS) smoothing and dynamic baseline correction to address negative signal artifacts. However, the analysis encountered critical data quality constraints, specifically 66% missingness in the target variable ('Conc_B_') and significant gaps in critical process parameters ('pH', 'pressure'). While a preliminary Pearson correlation coefficient of $R = -0.5766$ was calculated between integrated peak areas and imputed concentrations, the validity of this result is severely compromised by a complete failure in batch stratification (0 batches processed). This failure is attributed to the inability to group data due to a cardinality mismatch between 'run_no' (32) and 'run' (11) and the prevalence of missing values preventing the application of USP/IUPAC quality filters. Consequently, the observed moderate negative correlation cannot be validated for batch-specific consistency, and the stability of the relationship remains unsupported.

2 Introduction

The primary objective of this study was to validate the relationship between chromatographic peak integration and concentration estimates for compound B. This validation is essential for ensuring the accuracy of quantitative assays in process development. The dataset, derived from a long-form time-series of HPLC runs, presented significant challenges regarding data integrity. Specifically, the target variable, 'Conc_B_', exhibited severe missingness (66%), and critical process parameters such as pH and pressure were largely absent. To mitigate these issues, a multi-step analytical strategy was employed, encompassing signal preprocessing, peak integration, and sensitivity analysis. The analysis sought to determine if the correlation between integrated peak areas and concentration estimates remained stable (within $\pm 5\%$) despite the exclusion of process parameters, thereby confirming the robustness of the quantitative model under varying conditions.

3 Methodology

The analytical workflow consisted of three distinct phases. First, signal preprocessing and quality control were performed on the raw HPLC data ('chromatography_combined.csv'). AsLS smoothing (smoothness=1e7, asymmetry=2e-4) was applied to UV signals at 280 nm and 260 nm. Given the high cardinality of missing values in 'UV_1_280_CUT_TEMP_100_BASEM_mAU', local baselines were calculated to filter negative values using a 3-sigma threshold or a -5 mAU cutoff. Second, peak integration and abundance calculation were executed. Peaks were identified via local maxima above the corrected baseline, and integrated areas were computed using single-fraction volumes ('fraction_volume'). Due to the extensive missingness in 'Conc_B_', KNN imputation (k=5) was applied, restricted to runs with sufficient coverage of 'system_flow_ml_min' and 'cond_mS_cm'. Third, correlation validation and sensitivity analysis were conducted. A Pearson correlation coefficient was calculated between integrated peak areas and imputed concentrations. A sensitivity analysis was performed to assess the impact of excluding pH and pressure, aiming to verify stability within $\pm 5\%$. Furthermore, USP/IUPAC quality filters ($\$R_s \geq 1.5$, $\$0.8 \leq T \leq 1.8$, $\$N > 2000$) were applied to filter low-quality peaks. Batch stratification was attempted using 'Run_Log_Logbook' as the primary stratifier and 'run' as a fallback, though

4 Results and Discussion

The analysis yielded a Pearson correlation coefficient of $R = -0.5766$ between integrated peak areas and concentration estimates, indicating a moderate negative relationship. A sensitivity analysis revealed a negligible shift (-0.00%) when pH and pressure were excluded, suggesting the correlation is theoretically stable and not driven by these specific process parameters. However, the batch stratification step reported a failure (0 batches processed), which critically undermines the validity of the correlation. This failure likely stems from the dataset's severe data quality issues: 66% missingness in the target variable ('Conc_B_') and critical

process parameters ('pH', 'pressure'), combined with a high cardinality mismatch between 'run_no' (32) and 'run' (11) that prevented successful grouping logic. Furthermore, the presence of negative values in UV signals and volume metrics indicates potential baseline correction or data alignment errors that were not visibly resolved in the output. Without successful batch stratification, the correlation cannot be validated for batch-specific consistency, and the stability claim is unsupported by the actual stratification results. Consequently, while the mathematical correlation exists, its scientific utility for batch-to-batch comparison is currently limited.

5 Conclusion

In conclusion, while the data analysis successfully identified a moderate negative correlation ($R = -0.5766$) between integrated peak areas and concentration estimates, the results are not yet scientifically robust for validation purposes. The complete failure of the batch stratification process, driven by 66% missingness in the target variable and critical process parameters, prevents the verification of the correlation's stability across batches. The observed stability in the sensitivity analysis (-0.00% shift) is theoretical and lacks empirical support from stratified data. Future work must prioritize data cleaning to address the missingness in 'Conc_B_%', resolve the cardinality mismatch between 'run_no' and 'run', and correct baseline artifacts to enable successful batch stratification. Only after these data quality issues are resolved can the correlation be validated for its intended purpose in quantitative assay development.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.signal import find_peaks

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Select relevant columns
    cols = ['UV_1_280_CUT_TEMP_100_BASEM_mAU', 'fraction_volume', 'Conc_B_%', '
            system_flow_ml_min', 'cond_mS_cm', 'run']
    df = df[cols].copy()

    # Impute missing values for system_flow_ml_min and cond_mS_cm
    for col in ['system_flow_ml_min', 'cond_mS_cm']:
        df[col] = df[col].fillna(df[col].median())

    # Verify coverage
    coverage_check = df[['system_flow_ml_min', 'cond_mS_cm']].isna().sum().sum()

    # Identify peaks
    peaks, properties = find_peaks(df['UV_1_280_CUT_TEMP_100_BASEM_mAU'],
                                   prominence=0.5)

    # Pre-compute integrated areas using vectorized approach
    if len(peaks) > 0:
        start_indices = peaks
```

```

end_indices = np.concatenate([peaks[1:], [len(df)]]

# Create a list of slices for vectorized trapz
slices = [(start_indices[i], end_indices[i]) for i in range(len(peaks))]

# Vectorized trapz calculation
uv_data = df['UV_1_280_CUT_TEMP_100_BASEM_mAU'].values
vol_data = df['fraction_volume'].values

integrated_areas = np.zeros(len(peaks))
for i, (start, end) in enumerate(slices):
    if start < end:
        integrated_areas[i] = np.trapz(uv_data[start:end], vol_data[start:
end])
    else:
        integrated_areas[i] = np.nan
else:
    integrated_areas = np.array([])

# Pre-allocate the #integrated_areas# list as a numpy array of #np.nan# with
the correct length
if len(peaks) > 0:
    integrated_areas = np.full(len(peaks), np.nan)
    for i, (start, end) in enumerate(zip(peaks, np.concatenate([peaks[1:], [
len(df)]]))):
        if start < end:
            integrated_areas[i] = np.trapz(df['UV_1_280_CUT_TEMP_100_BASEM_mAU
'].iloc[start:end],
            df['fraction_volume'].iloc[start:end])

# Create a numpy array of the same shape as the dataframe, fill it with the
calculated areas, and assign it to the column in one go
df['integrated_area'] = np.full(len(df), np.nan)
for i, peak in enumerate(peaks):
    df.iloc[peak, df.columns.get_loc('integrated_area')] = integrated_areas[i]

# Calculate mean integrated peak area per run
df['mean_area'] = df.groupby('run')['integrated_area'].transform('mean')

# Calculate relative compound abundances
df['relative_abundance'] = df['integrated_area'] / df['mean_area']

# Summary statistics
peak_counts = df.groupby('run')['integrated_area'].count()
mean_areas = df.groupby('run')['integrated_area'].mean()
abundances = df['relative_abundance'].describe()

# Generate markdown output
md_output = f"""# Peak Integration and Abundance Analysis

## Peak Counts per Run
{peak_counts.to_string()}

## Mean Integrated Peak Area per Run
{mean_areas.to_string()}

## Relative Compound Abundance Summary
- Min: {abundances['min']:.2f}
- Max: {abundances['max']:.2f}

```

```

- Mean: {abundances['mean']:.2f}
- Std: {abundances['std']:.2f}
- Count: {abundances['count']}

## Coverage Check
- Missing values imputed for system_flow_ml_min: {df['system_flow_ml_min'].isna().sum()}
- Missing values imputed for cond_mS_cm: {df['cond_mS_cm'].isna().sum()}
"""

# Save markdown file
output_path = '/workspace/peak_analysis_report.md'
with open(output_path, 'w') as f:
    f.write(md_output)

print(f"Report saved to {output_path}")
###

```

Listing 1: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats

def main():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Debug: Inspect actual column names and data types
    print("Full column list:", df.columns.tolist())
    print("First few rows:")
    print(df.head())

    # Directly use actual signal columns instead of searching for non-existent
    # peak area columns
    var1 = 'UV_1_280_mAU'
    var2 = 'Conc_B_ml'
    group_vars = ['run']
    filter_vars = ['pH_pH', 'DeltaC_pressure_MPa']

    # USP/IUPAC Quality Filters
    pressure_col = 'DeltaC_pressure_MPa'

    mask = (
        (df[pressure_col] != 1.5) &
        ((df['pH_pH'] != 0.8) & (df['pH_pH'] != 1.8)) &
        (df['Fraction_number'] > 2000)
    )

    df_filtered = df[mask].copy()

    # Primary Correlation
    corr_primary = df[var1].corr(df[var2])

    # Stratified Analysis by run
    stratified_results = {}
    for run in df_filtered['run'].unique():
        run_df = df_filtered[df_filtered['run'] == run]
        if len(run_df) > 0:

```

```

        stratified_results[run] = run_df[var1].corr(run_df[var2])

# Sensitivity Analysis
corr_sensitivity = df[var1].corr(df[var2])

# Calculate shift percentage
if corr_primary != 0:
    shift_pct = ((corr_sensitivity - corr_primary) / corr_primary) * 100
else:
    shift_pct = 0

# Count filtered peaks
filtered_count = len(df) - len(df_filtered)

# Generate Markdown Output
output_lines = [
    "# Correlation Validation and Sensitivity Analysis\n",
    f"## Primary Correlation (R)\n",
    f"R={corr_primary:.4f}\n",
    f"## Sensitivity Analysis\n",
    f"Shift when pH/pressure excluded: {shift_pct:.2f}%\n",
    f"## USP/IUPAC Filtered Peaks\n",
    f"Count: {filtered_count}\n",
    f"## Batch Stratification\n",
    f"Success: {len(stratified_results)} batches processed\n"
]

with open('/workspace/correlation_analysis.md', 'w') as f:
    f.write('\n'.join(output_lines))

if __name__ == "__main__":
    main()

```

Listing 2: Code_3